

学号： 2106410120



沈阳建筑大学

大数据技术基础 作业

2023 年秋季学期

学 院： 计算机科学与工程学院

专业班级： 计算机 2101 班

姓 名： 魏露林

5. 抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

代码：

```
f = open('data.csv',mode='w',encoding='utf-8',newline='')
csv_writer=csv.DictWriter(f, fieldnames=[
    '标题',
    '评论',
    '推荐',
    '作者',
    '出版日期',
    '出版社',
    '售价',
    '折扣',
    '原价',
    '详情页',
])
csv_writer.writeheader()
for page in range(1,26):
    url = 'http://bang.dangdang.com/books/bestsellers/01.00.00.00.00.00-recent7-0-0-1-{page}'
    response = requests.get(url)
    print(response)
    # <Response [200]> 响应对象 200 状态码表示请求成功
    # 2. 获取数据，服务器返回响应数据 response.text 文本数据
    selector=parsel.Selector(response.text)
    lis=selector.css('.bang_list_mode li')
    # 取数据[1.get(取的是第一个数据，当 li 里有很多 div 时，提取可能发成错误),2.div:nth-child:
    # 用于指定 li 里的具体 div
    # 3..getall()[-1]:取全部]
    # lis=['a','b','c']
    # lis[2]
    # lis[-1]

    for li in lis:
        title = li.css('.name a::attr(title)').get() #标题
        comment = li.css('.star a::text').get() #评论数量
        recommend = li.css('.tuijian::text').get() #推荐指数
        author = li.css('.publisher_info a::attr(title)').get() #作者
        date = li.css('div:nth-child(6) span::text').get() #出版时间
        press = li.css('div:nth-child(6) a::text').get() #出版社
        price_n = li.css('div:nth-child(7) p:nth-child(1) .price_n::text').get() # 售价
        price_r = li.css('div:nth-child(7) p:nth-child(1) .price_r::text').get() # 原价
```

```
price_s = li.css('div:nth-child(7) p:nth-child(1) .price_s::text').get() # 折扣
href = li.css('.name a::attr(href)').get()
```

```
dit={
    '标题': title,
    '评论': comment,
    '推荐': recommend,
    '作者': author,
    '出版日期': date,
    '出版社': press,
    '售价': price_n,
    '折扣': price_s,
    '原价': price_r,
    '详情页': href,
}

csv_writer.writerow(dit)

print(title, comment, recommend, author, date, press, price_n, price_r, price_s, href)
```

程序分析：

- 1.数据源：程序访问了当当网站上的多个页面，每个页面包含了不同的畅销图书列表。
- 2.HTTP 请求：使用`requests.get()`方法发送 HTTP 请求，获取每个页面的 HTML 文本。
- 3.HTML 解析：程序使用 Parsel 库来解析 HTML 文本，以便能够轻松地选择和提取所需的数据。
4. 信息提取：从 HTML 文本中，程序通过 CSS 选择器选择了每本畅销图书的标题、评论数量、推荐指数、作者、出版日期、出版社、售价、折扣、原价以及详情页链接。
5. 数据保存：提取的图书信息被存储在一个 CSV 文件中，以便后续分析或其他用途。



6. 葡萄酒数据集 (wine.data) 搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

代码：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
# 读取数据集
data = pd.read_csv('wine.data', header=None)
# 将数据集分为特征和标签
X = data.iloc[:, 1:]
y = data.iloc[:, 0]
from sklearn.preprocessing import LabelEncoder
#XGBoost 要求目标变量必须是从 0 开始的整数，而 y 变量取值是 [1, 2, 3], 使用
LabelEncoder 进行转换
le = LabelEncoder()
xgy = le.fit_transform(y)
# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy, test_size=0.3, random_state=42)
# 建立决策树模型
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)
# 建立随机森林模型
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)
# 建立 xgBoost 模型
xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)
# 输出各个模型在测试集上的表现
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)
```

程序说明：

1. 导入必要的库：
2. 读取数据集：'wine.data'的 CSV 文件
3. 分离特征和标签：

-`X` 包含数据集的特征，通过选择所有行（:）和除第一列之外的所有列（1:）来实现。

-`y` 包含数据集的标签，通过选择所有行（:）和第一列（0）来实现。

4. 使用 LabelEncoder 对目标变量进行转换:

-XGBoost 要求目标变量必须是从 0 开始的整数，但原始数据中的 y 取值是[1, 2, 3]。因此，使用`LabelEncoder`将 y 进行转换，以生成`xgy`，其中 y 的取值变为[0, 1, 2]。

5. 将数据集分为训练集和测试集:

使用`train\_test\_split`函数将特征（X）和原始标签（y）分为训练集和测试集，测试集的大小为 30%，并设置了一个随机种子（random_state=42）以确保可复现性。

6. 建立决策树模型

7. 建立随机森林模型: 创建一个`RandomForestClassifier`对象

8. 建立 XGBoost 模型: 创建一个`XGBClassifier`对象到`xgb\_score`作为分数。

9. 输出各个模型在测试集上的表现:

打印决策树模型、随机森林模型和 XGBoost 模型在测试集上的得分（准确性）。

运行截图

